

Практическая работа по Информатике

Указатели. Односвязный список

Указатель – это переменная, в которой хранится адрес. В Паскале обычно применяют типизированные указатели – переменные, хранящие адрес, относительно которого точно известно, переменная какого типа расположена в соответствующей области памяти.

Для обозначения указателя в оригинальном виртовском описании Паскаля использовался символ стрелки «↑», но на нашей клавиатуре такого символа нет, поэтому во всех версиях Паскаля используется символ «^», который расположен над цифрой 6. Например, если описать три переменные:

```
var
  p: ^integer;
  r: real;
  q: ^real;
begin
  q := @r;
```

то **p** сможет хранить адрес переменной, имеющей тип `integer`, **q** – адрес переменной `real`, а **r** – переменная типа `real`.

Адрес переменной всегда можно получить с помощью операции взятия адреса **@**. В первом примере мы взяли адрес переменной **r** и занесли его в переменную **q**, т.е. **q** теперь хранит адрес **r**.

Для обращения к переменной через указатель или обращения к переменной по адресу также используется символ стрелки, которая ставится после имени указателя, например, оператор

```
p^ := 13;    { r := 13 }
```

занесёт в память, находящуюся по адресу, хранящемуся в **p**, значение 13.

Результатом операции взятия адреса является нетипизированный адрес. Для изменения этого поведения компилятора есть директива `{ST+}`.

Встроенная константа **nil** обозначает недействительный адрес, по которому в памяти заведомо не может находиться ни одна переменная. Константа **nil** – специальный «нулевой адрес». Эту константу присваивают переменным указательных типов, чтобы показать, что данный указатель сейчас никуда не указывает.

Динамическая переменная – это область памяти, которая выделяется во время работы программы. Работа с динамическими переменными производится с использованием адресов, хранящихся в указателях. Отметим некоторые аспекты при работе с динамическими переменными:

- Память под динамические переменные выделяется из специальной области, называемой *кучей* (англ. *heap*).
- При уничтожении динамической переменной память возвращается обратно в «кучу».
- При необходимости наша программа обращается к операционной системе с просьбой выдать больше памяти, за счёт чего увеличивает размер кучи.
- Уменьшить размер кучи невозможно, поэтому надо сначала удалить старую динамическую переменную, а затем уже создавать новую.

Для создания динамической переменной служит псевдопроцедура **new**, которая должна быть применена к типизированному указателю:

```
var
  p: ^string;
```

```
begin    new(p);
```

- 1) из кучи будет выделена область памяти размером 256 байт;
- 2) в переменную p будет занесён адрес этой области памяти.

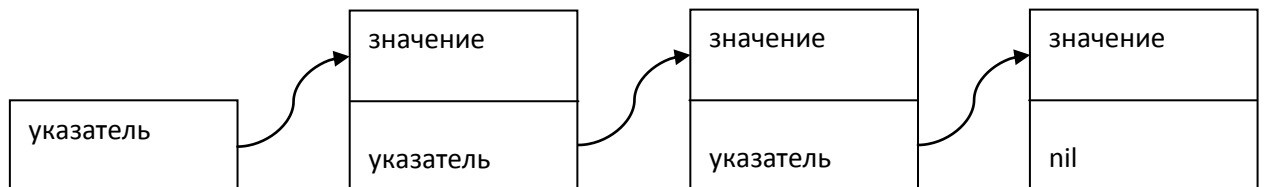
Удаление динамической переменной производится с помощью псевдопроцедуры **dispose**:

```
dispose(p);
```

При `dispose(p)` значение указателя p не меняется; в кучу возвращается память, на которую указывал p, т.е. эта область памяти помечается свободной; её можно выдать по требованию одним из следующих **new**.

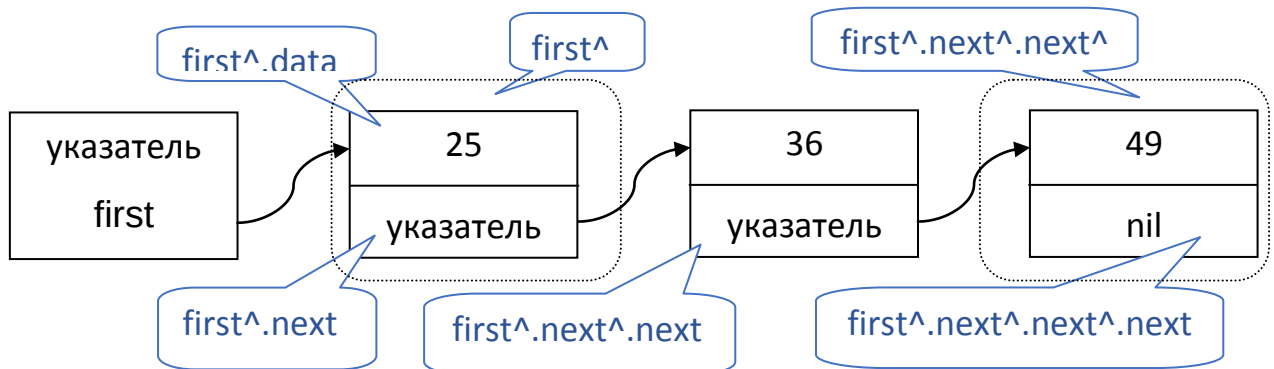
Наиболее полно возможности указателей раскрываются только при создании так называемых **связных динамических структур данных**, которые состоят из отдельных переменных типа «запись», причём каждая такая переменная содержит один или несколько указателей на другие переменные того же типа.

Пример **односвязного списка** из трёх элементов, обобщённая диаграмма:



Слово **односвязный** означает, что на каждый из элементов списка указывает один указатель.

На следующей диаграмме также изображён односвязный список из трёх элементов, но теперь в него занесены конкретные значения:



`first` – указатель на 1-й элемент;

`first^` - весь элемент целиком;

`first^.data` – поле 1-го элемента, в котором содержится значение – в примере это число 25;

`first^.next` – указатель на следующий, второй, элемент списка;

`first^.next^.data` – поле 2-го элемента со значением 36;

`first^.next^.next` – указатель на третий элемент списка;

`first^.next^.next^` - весь третий элемент целиком и т.д.

Для создания такого списка нам потребуется запись, состоящая из двух полей: первое будет хранить целое число, второе – адрес следующего звена списка. Здесь в Паскале имеется одна

особенность: использовать имя описываемого типа при описании его собственных полей нельзя, но Паскаль позволяет описать указательный тип, используя такое имя типа, которое пока ещё не введено:

```
type
  itemptr = ^item;
  item = record
    data: integer;
    next: ^item;
  end;
var
  first: itemptr;
begin
  new(first);
  first^.data := 25;
  new(first^.next);
  first^.next^.data := 36;
  new(first^.next^.next);
  first^.next^.next^.data := 49;
  first^.next^.next^.next := nil;
  dispose(first^.next^.next);
  dispose(first^.next);
  dispose(first)
end.
```

В примере выше приведён порядок создания списка из трёх элементов, а затем сразу же произведено удаление этих динамических данных во избежание появления так называемого **мусора** в памяти или **утечек памяти**.

Данный пример приведён только для иллюстрации и наглядного изображения элементов односвязного списка; в реальности обычно не используются обращения к элементам списка с помощью громоздких конструкций `first^.next^.next^.data` и `first^.next^.next^.next`

Задание 1.

Написать программу, которая читает целые числа из потока стандартного ввода (клавиатуры) до возникновения ситуации «конец файла» (имитируется нажатием Ctrl+D), после чего печатает все введённые числа в обратном порядке. Количество чисел заранее неизвестно, вводить явные ограничения на это нельзя. Используйте односвязный список и добавляйте элементы в его начало.

Пример **ввода** и **вывода** (^D – это нажатие Ctrl+D):

```
1
2
3
^D
3
2
1
```

Продолжение на следующей странице

Задание 2.

Написать программу, которая читает целые числа из потока стандартного ввода (клавиатуры) до возникновения ситуации «конец файла» (имитируется нажатием Ctrl+D), после чего **дважды** печатает все введённые числа в том порядке, в котором они были введены. Количество чисел заранее неизвестно, вводить явные ограничения на это нельзя. Используйте односвязный список, но добавляйте новые элементы в его конец; для этого придётся задействовать два указателя: первый будет хранить адрес первого элемента списка, второй – адрес последнего элемента.

Пример **ввода** и **вывода** (^D – это нажатие Ctrl+D):

```
1
2
3
^D
1
2
3
1
2
3
```